Memoria de Práctica: Laboratorio 1 - Ingesta de Fuentes Heterogéneas

Módulo: Sistemas de Big Data (IABD)

Alumno: Ángel Postigo Rodríguez

1. Parte A: Ingesta BATCH con Sqoop

En esta primera parte vamos a traspasar la tabla histórica de pedidos desde una base de datos MySQL hacia el sistema de archivos HDFS usando Apache Sqoop.

Paso A1: Preparación del contenedor MySQL y exploración de la fuente (1.000 registros)

Antes de iniciar la extracción masiva hacia HDFS, se verificó la disponibilidad y el volumen de los datos en el sistema transaccional de origen (MySQL). Se generó la tabla de pedidos con 1.000 registros de prueba. Para validarlo, se accedió al contenedor origen y se ejecutaron los siguientes comandos SQL de auditoría:

```
USE ventas_db;
SHOW TABLES;
SELECT COUNT(*) AS total_pedidos FROM pedidos;
```

(El resultado de la consulta confirmó la presencia de los 1.000 registros, lo cual concuerda con el tamaño posterior del bloque en HDFS de 24.803 bytes).

Comando de Ingesta (Sqoop)

```
sqoop import \
--connect jdbc:mysql://mysql/ventas_db \
--username sqoopuser \
--password 'sqooppassword' \
--table pedidos \
--target-dir /user/iabd_admin/ingesta/ventas/pedidos \
--m 1
```

Evidencias de Ejecución (Sqoop)

```
bash-4.2$ hdfs dfs -ls /user/iabd_admin/ingesta/ventas/pedidos
Found 2 items
-rw-r--r--   1 hadoop supergroup          0 2026-11-04 08:26 /user/iabd_admin/ingesta/ventas/pedidos/_SUCCESS
-rw-r--r--   1 hadoop supergroup      24803 2026-11-04 08:26 /user/iabd_admin/ingesta/ventas/pedidos/part-m-00000
bash-4.2$
```

```
PS C:\Users\angel\master-ia-bigdata-master> docker exec -it namenode bash
bash-4.2$ hdfs dfs -cat /user/iabd_admin/ingesta/ventas/pedidos/part-m-00000
1,Producto_1,5,49.75
2,Producto_2,2,44.59
3,Producto_3,7,99.41
4,Producto_4,10,87.44
5,Producto_5,5,66.74
6,Producto_6,10,81.03
7,Producto_7,2,38.88
8,Producto_8,5,9.42
9,Producto_9,2,29.07
10,Producto_10,2,67.52
11,Producto_11,1,21.26
12,Producto_12,10,96.25
13,Producto_13,1,37.37
14,Producto_14,8,43.81
15,Producto_15,1,95.99
16,Producto_16,7,29.48
17,Producto_17,6,96.31
18,Producto_18,2,64.50
19,Producto_19,10,58.73
20,Producto_20,3,35.43
21,Producto_21,2,44.61
22,Producto_22,10,28.70
23,Producto_23,7,43.78
24,Producto_24,3,73.59
25,Producto_25,1,4.14
26,Producto_26,1,16.38
27,Producto_27,7,73.95
28,Producto_28,8,57.67
29,Producto_29,7,30.11
30,Producto_30,7,53.08
31,Producto_31,6,37.57
32,Producto_32,1,36.16
33,Producto_33,6,48.90
34,Producto_34,10,3.57
35,Producto_35,5,27.71
36,Producto_36,10,94.32
37,Producto_37,9,42.68
38,Producto_38,6,62.13
39,Producto_39,4,96.28
40,Producto_40,8,70.21
41,Producto_41,4,67.53
42,Producto_42,4,51.06
43,Producto_43,7,62.46
44,Producto_44,3,27.23
45,Producto_45,7,55.18
46,Producto_46,8,3.40
47,Producto_47,10,67.91
48,Producto_48,6,59.63
49,Producto_49,5,21.85
50,Producto_50,9,78.43
51,Producto_51,3,93.58
52,Producto_52,10,72.17
53,Producto_53,9,29.03
```

2. Parte B: Ingesta STREAMING con Flume

Aquí configuramos un agente de Apache Flume para capturar de manera continua los registros ficticios de un servidor web, enviándolos directamente desde una carpeta local hacia HDFS.

Archivo de Configuración (flume-conf.properties)

```
a1.sources = r1
a1.channels = c1
a1.sinks = k1

a1.sources.r1.type = spooldir
a1.sources.r1.spoolDir = /data/flume/spool
a1.sources.r1.fileHeader = true
a1.sources.r1.channels = c1

a1.channels.c1.type = memory
a1.channels.c1.capacity = 1000
a1.channels.c1.transactionCapacity = 100

a1.sinks.k1.type = hdfs
a1.sinks.k1.hdfs.path                                                    =
hdfs://namenode:8020/user/iabd_admin/ingesta/streaming/ventas
a1.sinks.k1.hdfs.fileType = DataStream
a1.sinks.k1.hdfs.writeFormat = Text
a1.sinks.k1.hdfs.rollInterval = 60
a1.sinks.k1.hdfs.rollSize = 0
a1.sinks.k1.hdfs.rollCount = 0
a1.sinks.k1.hdfs.useLocalTimeStamp = true
a1.sinks.k1.channel = c1
```

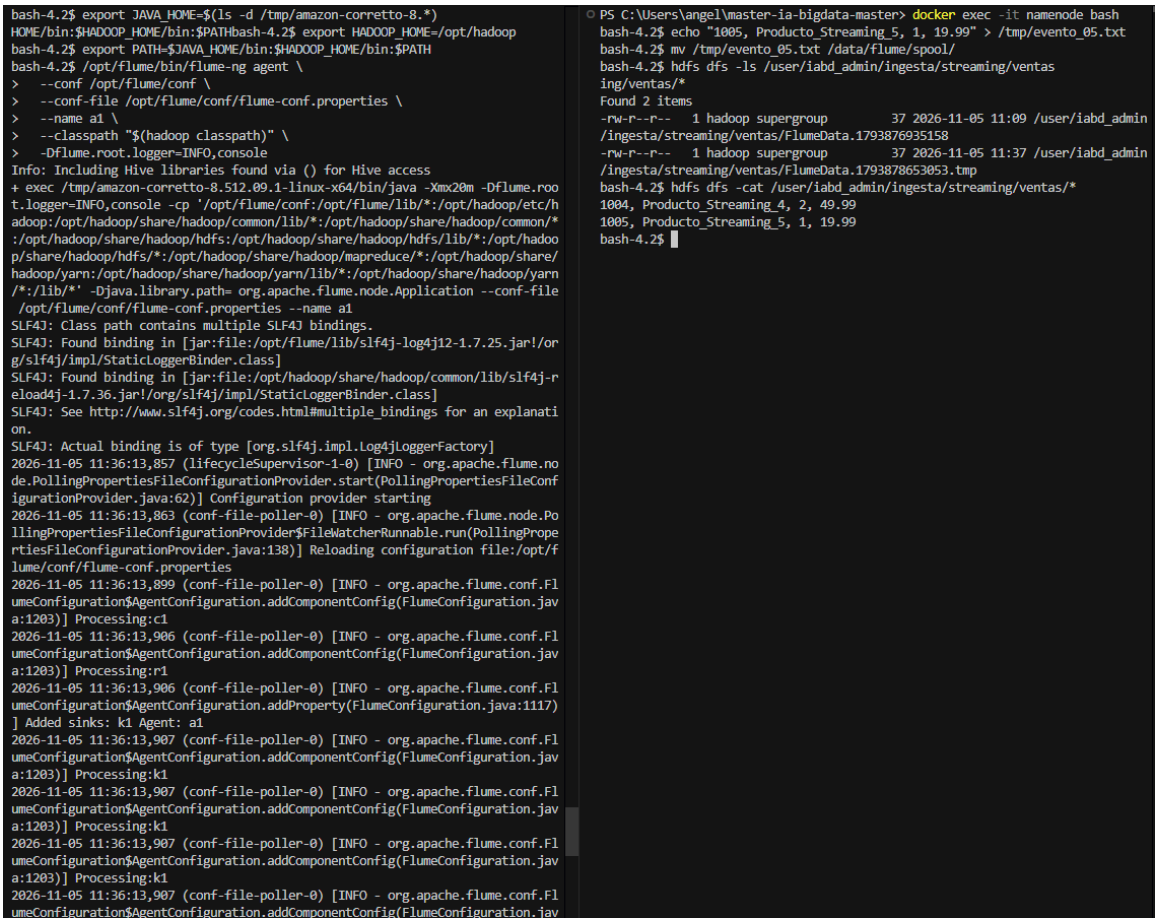
Evidencias de Ejecución (Flume)

Preparación del entorno y comando de arranque

Para asegurar el correcto funcionamiento del agente en el NameNode, se inicializó la ruta local monitorizada y se levantó el servicio de Flume inyectando dinámicamente el Classpath de Hadoop:

```
mkdir -p /data/flume/spool
flume-ng agent \
  --conf-file /opt/flume/conf/flume-conf.properties \
  --name a1 \
  -Dflume.root.logger=INFO,console \
  --classpath "$(hadoop classpath)"
```

Nota de arquitectura: Al ejecutarse sobre un clúster contenerizado en Docker, el endpoint HDFS del sink apunta al FQDN hdfs://namenode:8020 en lugar del clásico localhost:9000, y se emplea la ruta /data/flume/spool para persistir el directorio de spooling montado en el contenedor.



Justificación del payload de Streaming (Paso B5)

Como se demuestra en la evidencia gráfica anterior, se generaron y movieron dos eventos simulados mediante el comando mv (evento_04 y evento_05). La inspección de HDFS revela el ciclo de vida del sink: se observa un fichero de log ya cerrado y consolidado tras vencer el tiempo de rotación (FlumeData.1793876035158) y un segundo fichero actualmente abierto en escritura con el sufijo temporal en curso (FlumeData.1793878653053.tmp), confirmando el correcto funcionamiento del parámetro rollInterval = 60.

3. Discusión y Cierre

¿Por qué no usamos Sqoop para los logs? ¿Qué pasaría?

Sqoop está pensado para mover volúmenes grandes de datos estructurados de una sola vez (modo Batch) desde bases de datos relacionales. Los registros de logs, en cambio, llegan de forma continua y no tienen una estructura tan rígida. Si intentamos usar Sqoop para esto, tendríamos que lanzarlo a cada momento en pequeños bloques. Esto sobrecargaría el sistema al iniciar tareas de MapReduce constantemente, generando mucha lentitud y un consumo de recursos innecesario, además del peligro de perder eventos que no estén guardados todavía.

¿Por qué no usamos Flume para la base de datos? ¿Se podría?

Aunque existen conectores que permiten a Flume hacer consultas periódicas a una base de datos, no es la mejor opción. Flume no está diseñado para mantener las relaciones ni la consistencia propia de una base de datos transaccional, ni para dividir la carga de trabajo en paralelo como sí lo hace Sqoop. Además, hacer consultas constantes a tablas muy grandes terminaría ralentizando el rendimiento de la propia base de datos.

En Flume, usamos un memory channel. ¿Qué riesgo tiene esto en un entorno de producción real?

El mayor riesgo es perder la información si ocurre un problema. Al guardar los datos en la memoria RAM (que es volátil), si el agente de Flume se detiene por un fallo, falta de memoria o un reinicio del sistema, todos los registros que estaban en espera y aún no se habían escrito en HDFS se borrarán por completo. En un caso real de producción, lo ideal sería usar un canal en disco ("file channel") o apoyarse en herramientas como Apache Kafka para garantizar que los datos no se pierdan.